

Platform Optimization Checklist

34 actionable items across memory management, connection pooling, deployment hardening, and performance monitoring — prioritized by severity and timeline.

7

CRITICAL

9

HIGH

12

MEDIUM

6

PREVENTIVE

Stop the Bleeding

Critical deployment and runtime fixes that can be done today with zero code changes to the Django app.

☐ Replace `manage.py runserver` with Gunicorn

Django's dev server has known memory leaks in long-running processes. Gunicorn with `--max-requests` auto-recycles workers, preventing unbounded memory growth.

CRITICAL

DAY 0

Dockerfile ~40% memory reduction

```
# Dockerfile – Replace CMD
RUN pip install gunicorn
CMD ["gunicorn", "LinkedEyeWebProject.wsgi:application", \
    "--bind", "0.0.0.0:8000", \
    "--workers", "4", \
    "--worker-class", "sync", \
    "--max-requests", "1000", \
    "--max-requests-jitter", "50", \
    "--timeout", "120", \
    "--graceful-timeout", "30", \
    "--preload"]
```

☐ Set K8s memory limits + restart policy

Without resource limits, a leaking container can OOM the entire node. Set hard limits so K8s auto-restarts before damage spreads.

CRITICAL

DAY 0

k8s/deployment.yaml

```
resources:
  requests:
    memory: "512Mi"
    cpu: "500m"
  limits:
    memory: "1536Mi"
    cpu: "2000m"
livenessProbe:
  httpGet:
    path: /health/
    port: 8000
  initialDelaySeconds: 30
  periodSeconds: 15
  failureThreshold: 3
```

☐ Add health check endpoint

Minimal Django view that returns 200 — required for K8s probes. Don't hit the database; just confirm the process is alive.

CRITICAL

DAY 0

app/views.py + urls.py

```
# app/views.py
def health_check(request):
    proc = psutil.Process(os.getpid())
    return JsonResponse({
        "status": "ok",
        "rss_mb": round(proc.memory_info().rss / 1024 / 1024, 1),
        "threads": threading.active_count(),
        "connections": len(proc.connections()),
    })

# urls.py
urlpatterns += [path("health/", health_check)]
```

PHASE 2 – WEEK 1

Fix Connection Leaks

Neo4j driver accumulation is the #1 cause of memory growth. Fix the three biggest connection leak sources.

☐ Neo4j: Singleton driver with connection pool

Currently, every `Node()` call creates a new Bolt driver with its own TCP connection pool. 15+ views do this per request. Replace with a single module-level driver.

CRITICAL

WEEK 1

lib/LinkedEyeEntity/Node.py ~1-10MB saved per request

```
# Module-level singleton
_driver_lock = threading.Lock()
_driver = None

def get_driver():
    global _driver
    if _driver is None:
        with _driver_lock:
            if _driver is None:
                _driver = GraphDatabase.driver(
                    settings.NEO4J_BOLT_URL,
                    auth=(settings.NEO4J_USER, settings.NEO4J_PASS),
                    max_connection_pool_size=25,
                    connection_acquisition_timeout=30,
                )
    return _driver
```

☐ Replace `threading.Timer` with Celery delayed tasks

Each snooze creates a new thread (~8MB stack). Replace with Celery `apply_async(countdown=N)` which uses the existing RabbitMQ worker — zero extra threads.

CRITICAL

WEEK 1

notification/views.py → notification/tasks.py ~8MB saved per snooze

```
# BEFORE:
timer = threading.Timer(snooze_seconds, resume_notifications, args=[...])
timer.start()

# AFTER:
resume_notifications.apply_async(
    args=[service_id, user_id],
    countdown=snooze_seconds
)
```

☐ RabbitMQ: Connection pool or publish-and-close pattern

Each `MQ()` creates a pika BlockingConnection + a hidden Node() instance. Ensure `close()` is always called via context manager.

HIGH

WEEK 1

lib/LinkedEyeMQ/MQ.py

```
class MQ:
    def __enter__(self): return self
    def __exit__(self, *args): self.close()

# Usage:
with MQ() as mq:
    mq.publish(exchange, message)
```

☐ Remove Node() instantiation from MQ.py and Redis.py

Both MQ.py and Redis.py create hidden Node() instances internally, causing cascading Neo4j connection leaks even on non-graph operations.

CRITICAL

WEEK 1

lib/LinkedEyeMQ/MQ.py, lib/LinkedEyeRedis/Redis.py

☐ Parameterize ALL Cypher queries (stop injection)

Every query method in Node.py uses string concatenation with user input. Use Neo4j parameters:

```
session.run("MATCH (n {title: $t})", t=title)
```

CRITICAL

WEEK 1

lib/LinkedEyeEntity/Node.py (all methods)

SECURITY

PHASE 3 – WEEK 2

Database & Service Connection Pooling

Every analytics request leaks PostgreSQL, MySQL, and Elasticsearch connections. Fix with pooled singletons.

☐ PostgreSQL: Use `django-db-connection-pool` or PgBouncer

psycopg2 connections in analytics/views.py are created raw per request. Either use Django's ORM with pooling, or put PgBouncer in front.

HIGH

WEEK 2

analytics/views.py, settings.py

```
DATABASES['superset'] = {
    'ENGINE': 'dj_db_conn_pool.backends.postgresql',
    'POOL_OPTIONS': {
        'POOL_SIZE': 5,
        'MAX_OVERFLOW': 10,
        'RECYCLE': 300,
    }
}
```

☐ Elasticsearch: Singleton client with sniffing

New ES client per request = new urllib3 pool per request. Create one module-level client in `lib/es_client.py`.

HIGH

WEEK 2

analytics/views.py

```
es_client = Elasticsearch(
    [settings.ES_HOST],
    timeout=30,
    max_retries=3,
    retry_on_timeout=True,
    maxsize=10,
)
```

☐ MySQL analytics: Use Django ORM or connection pool

messagequeue_views.py creates raw mysql.connector connections. Route through Django's ORM or add `try/finally` close pattern.

HIGH

WEEK 2

bodeodstatus/messagequeue_views.py

☐ Add pagination to ALL unbounded queries

auditlogs loads ALL logs. analytics loads 10k ES records. Node.overviewStats() runs 16 queries per IP with no limit. Add `LIMIT` and pagination.

HIGH

WEEK 2

auditlogs/views.py, analytics/views.py, Node.py

☐ Enable Django Redis cache backend

Redis is already in the stack but not used for Django caching. Add it as cache backend for sessions, repeated queries, and template fragments.

HIGH

WEEK 2

settings.py

```
CACHES = {
    "default": {
        "BACKEND": "django_redis.cache.RedisCache",
        "LOCATION": "redis://redis.fs-linkedeye:32268/1",
        "KEY_PREFIX": "le",
    }
}
SESSION_ENGINE = "django.contrib.sessions.backends.cache"
```

☐ Fix Apprise URL accumulation

Create a fresh Apprise instance per notification cycle instead of reusing one that keeps adding URLs without clearing.

MEDIUM

WEEK 2

lib/LinkedEyeNotification/Notification.py

PHASE 4 — WEEK 3

State Management & Architecture Cleanup

Remove global mutable state, eliminate duplicate apps, and add structured logging.

☐ Remove global mutable state: `Obj = {}` in vault/views.py

Module-level `Obj = {}` persists across ALL requests for the entire process lifetime. Replace with request-scoped or cached Vault client.

HIGH

WEEK 3

vault/views.py

☐ Replace `ast.literal_eval()` with `json.loads()`

Redis.py and Vault.py use `ast.literal_eval` on external data — a code execution risk. `json.loads` is safe and faster.

HIGH

WEEK 3

lib/LinkedEyeRedis/Redis.py, lib/LinkedEyeVault/Vault.py

SECURITY

☐ Merge duplicate apps: sites/lesites, allonboard/newonb

Two pairs of nearly identical apps double the maintenance surface and connection footprint. Consolidate into single apps.

MEDIUM

WEEK 3

sites/ + lesites/, allonboard/ + newonb/

☐ Add Django LOGGING config (replace print statements)

No LOGGING in settings.py. print() statements go to Docker log buffer without structure. Add JSON structured logging.

MEDIUM

WEEK 3

settings.py

```
LOGGING = {
    'version': 1,
    'handlers': {'console': {'class': 'logging.StreamHandler', 'formatter': 'json'}},
    'formatters': {'json': {'class': 'pythonjsonlogger.jsonlogger.JsonFormatter'}},
    'root': {'level': 'INFO', 'handlers': ['console']},
}
```

☐ Move hardcoded credentials to environment variables

10+ hardcoded passwords found in source (Neo4j, MySQL, SMTP, admin). Move all to K8s secrets + env vars.

CRITICAL

WEEK 3

Node.py, settings.py, notification/views.py, login/views.py

SECURITY

☐ Fix file handles: use with statements everywhere

Notification.py, Node.py, and Vault.py open files without context managers. Leaked file descriptors accumulate over time.

MEDIUM

WEEK 3

Notification.py:71, Node.py:635, Vault.py

☐ Streaming for large exports (Excel/PDF)

Analytics exports load entire datasets into openpyxl Workbook objects in memory. Use streaming write mode or chunked generation.

MEDIUM

WEEK 3

analytics/views.py

PHASE 5 – WEEK 4

Monitoring & Preventive Controls

Add runtime memory monitoring, CI guardrails, and soak testing to catch regressions before they hit production.

☐ Add MemoryGuardMiddleware

Django middleware that tracks RSS per request, warns at 768MB, forces GC at 1024MB. Logs memory stats to Elasticsearch for dashboarding.

MEDIUM

WEEK 4

LinkedEyeWebProject/middleware.py

```
class MemoryGuardMiddleware:
    WARN_MB = 768
    CRIT_MB = 1024

    def __call__(self, request):
        response = self.get_response(request)
        rss = self.proc.memory_info().rss / 1024 / 1024
        if rss > self.CRIT_MB:
            logger.critical(f"RSS={rss:.0f}MB threads={threading.active_count()}")
            gc.collect()
        elif rss > self.WARN_MB:
            logger.warning(f"RSS={rss:.0f}MB")
        return response
```

☐ CI guardrails: Static analysis for connection leaks

GitHub Actions workflow that greps for unmanaged connections, threading.Timer usage, global mutable state, and eval() on every PR.

MEDIUM

WEEK 4

.github/workflows/memory-check.yml

☐

8-hour soak test protocol

Run locust/k6 load test simulating market-day traffic for 8 hours. Track RSS growth rate. Pass criteria: <20MB/hr growth.

MEDIUM

WEEK 4

tests/soak_test.py

☐

Prometheus metrics for LinkedEye process health

Expose /metrics endpoint with process_rss_bytes, active_threads, open_connections, neo4j_pool_size, request_latency_p99.

MEDIUM

WEEK 4

LinkedEyeWebProject/metrics.py

RSS > 1024MB

Alert + force GC

Growth > 20MB/hr

Alert + investigate

Threads > 20

Timer leak warning

TCP conns > 100

Connection leak

GC gen2 > 100/min

Reference cycles

P99 latency > 5s

Performance alert

☐

K8s HPA for auto-scaling during market hours

Scale linkedeye-web pods from 2→5 based on CPU/memory during 09:00-15:30 IST (trading hours).

MEDIUM

WEEK 4

k8s/hpa.yaml

```
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef:
    name: linkedeye-web
  minReplicas: 2
  maxReplicas: 5
  metrics:
  - type: Resource
    resource: {name: cpu, target: {type: Utilization, averageUtilization: 70}}
  - type: Resource
    resource: {name: memory, target: {type: Utilization, averageUtilization: 80}}
```

☐

Extract service layer from views.py

Business logic is embedded in Django views. Move to services.py per app for testability, reuse, and cleaner connection management.

PREVENTIVE

ONGOING

All Django apps

EXPECTED IMPACT

Optimization Summary

Estimated improvement after completing all 5 phases.

60-80%

Memory reduction (RSS)

90%+

Connection leaks eliminated

<20MB/hr

Target RSS growth rate

4 weeks

Full implementation timeline